

Pocket Reference Manual v1.0.0

README

Pocket 🍷 — Five Native Utilities, Four Platforms

The five tools

Platforms

Tool × Platform matrix

Architecture at a glance

🔍 Observability (free tier, off-by-default)

🚀 Run It

Roadmap (post-v1)

License

CHANGELOG

Changelog

[Unreleased] - v0.1.0

Commit history (since repo creation)

DESIGN

DESIGN — Pocket

Repo map

5-tool catalogue (sensor stack per platform)

Layered design

Why Apple Watch + Wear OS skip Measure

Why combined permissions in onboarding

OBSERVABILITY

Pocket — Observability

Stack

How the stubs work

Privacy contract

How to drop in real SDKs (post-launch)

Alternatives matrix

Local development

PRIVACY

Pocket — Privacy Policy

TL;DR

What Pocket stores

What Pocket sends off-device (only if you opt in)

What we never collect

Permissions we ask for

Children

Changes to this policy

Contact

Where this policy is hosted

PRODUCTION

Pocket — Production Readiness

Gap audit

Free-deploy path (today)

One-time setup before tagging `v1.0.0`

3-week pre-launch polish list

Is it ready today?

QUICKSTART

Pocket — 5-Minute Quick-Start

🌐 Marketing site — 30 seconds

🍏 iOS app — 5 minutes (macOS only)

🕒 watchOS app — 5 minutes (macOS only)

📱 Android phone app — 5 minutes

🕒 Wear OS app — 5 minutes

🔧 Run every test on your machine — 1 line

🚀 Cut a release — 2 lines

🆘 Help

RELEASE-VIDEO-SCRIPT

Release Video Script — Pocket
Hero — top of marketing site | 4
Features overview | 5
Highlight reel | 5
What's next | 5
Call to action — bottom of page | 4

RELEASING

Pocket — Releasing

TL;DR

Tag conventions

What `bump-version.sh` touches

One-time setup per remote target

Local dry-run

Rollback

Changelog

SENTRY

Pocket — Sentry

Wiring per platform

DSN secrets

Privacy posture

Verifying

Self-hosting

Related docs

SIGNING

Signing — Pocket

Android

iOS

Desktop

watchOS

STORE-PACKAGING

Pocket — Store Packaging

1. Apple Watch bundling reality check

2. iOS bundle IDs

3. Android packaging

4. Marketing site

5. App icons

6. Localizations

Desktop binaries (Electron)

TESTING

Pocket — Testing

Sanity-test matrix per platform

Automated test matrix (CI)

Real-device-only checklist

Known caveats

Run every test on your machine

TOOLS-FEATURES

TOOLS-FEATURES — Pocket

Cross-tool matrix

1. Clock

2. Calculator

3. Measure

4. Compass

5. Level

AMERICAN GROUP LLC

Pocket

Reference Manual

Version 1.0.0

Generated 2026-05-08

README

README.md

Pocket 🍂 — Five Native Utilities, Four Platforms

 **Docs:** [QUICKSTART](#) · [DESIGN](#) · [TESTING](#) · [RELEASING](#) · [PRODUCTION](#) · [STORE-PACKAGING](#) · [SENTRY](#) · [OBSERVABILITY](#) · [TOOLS-FEATURES](#) · [PRIVACY](#)

Five tools that disappear into the OS. Clock · Calculator · Measure · Compass · Level — one app, four native UIs, zero account, no tracking by default.

The five tools

- 1. **Clock** — alarms, world clock, stopwatch, timer, bedtime. Real OS-scheduled alarms.
- 2. **Calculator** — basic + scientific (rotate to landscape on iPhone).
- 3. **Measure** — ARKit on iOS, ARCore on Android, on-screen ruler fallback.
- 4. **Compass** — magnetic + true heading, lat/lon, accuracy ring.
- 5. **Level** — flat-surface bullseye + tilted-surface bubble.

Platforms

Platform	Folder	Stack	Bundle ID
 iPhone	ios/	Swift + SwiftUI + ARKit (iOS 17+)	<code>com.americangroupllc.pocket</code>
 Apple Watch	watchos/	Swift + SwiftUI + WidgetKit (watchOS 10+)	<code>com.americangroupllc.pocket (.complication for widget)</code>
 Android phone	android/app/	Kotlin + Compose + Hilt + Room + ARCore (API 24+)	<code>com.americangroupllc.pocket</code>
 Wear OS	android/wear/	Kotlin + Wear Compose + Tile + Complication (API 30+)	<code>com.americangroupllc.pocketwear</code>

After renaming the GitHub repo to `Pocket`, update `index.html` canonical + OG URLs and `sitemap.xml`. The marketing site will continue to be served from `/ClockApp/` until the repo rename.

Plus the public-facing **marketing site** (`index.html` / `styles.css` / `script.js`) at the repo root.

Tool × Platform matrix

	Clock	Calculator	Measure	Compass	Level
iPhone	✓	✓ basic + scientific	✓ ARKit	✓ + lat/lon	✓ flat + tilted
Apple Watch	✓	✓ basic	—	✓ heading-only	✓ flat
Android phone	✓	✓ basic + scientific	✓ ARCore	✓ + lat/lon	✓ flat + tilted
Wear OS	✓ (no on-device alarms)	✓ basic	—	✓ heading-only	✓ flat

Wear OS skips on-device alarms by design — phone alarms surface on the watch via the standard companion bridge. Measure is phone-only (no AR on watches).

Per-tool capabilities, sensor stacks, and data flows: [TOOLS-FEATURES.md](#).

Architecture at a glance

```

Pocket/
├── shared/PocketCore/  Swift Package (iOS + watchOS): Clock + Calculator +
                        Compass + Level domain logic + Analytics/Crash stubs.
├── ios/                SwiftUI iPhone app (XcodeGen) – 5 tools.
├── watchos/            SwiftUI Apple Watch app (XcodeGen) – 4 tools +
                        WidgetKit complication.
└── android/            Multi-module Gradle project:
    ├── core/           :core  Kotlin library: same 5-tool domain logic,
    │                   obs stubs, mirrored tests.
    ├── app/            :app   Phone Compose app + Hilt + Room + AlarmManager +
    │                   BootReceiver + ARCore (Measure) + sensor wrappers.
    └── wear/           :wear   Wear Compose app + Tile + Complication services.

```

Each platform uses its idiomatic UI framework and language. Apple platforms share a Swift Package (`PocketCore`); Android phone + wear share a Gradle module (`:core`).

Observability (free tier, off-by-default)

Concern	Tool	Free tier	Wired in
Crashes / APM	Sentry	5K errors/mo	iOS · watchOS · Android · Wear
Product analytics	PostHog	1M events/mo (OSS, EU)	iOS · Android

Privacy contract: every SDK is **off by default**. Users opt in via Settings → Privacy. Wrappers strip user identifiers and never send tool inputs (no calculator history, no headings, no AR measurements, no alarm names). Real install steps in [OBSERVABILITY.md](#) and [SENTRY.md](#).

Run It

iOS (iPhone)

```

brew install xcodegen
cd ios && xcodegen generate && open Pocket.xcodeproj

```

Pick an iPhone 15 simulator + . See [ios/README.md](#) and [ios/SIGNING.md](#).

watchOS (Apple Watch)

```

brew install xcodegen
cd watchos && xcodegen generate && open PocketWatch.xcodeproj

```

Android (phone)

```

cd android
gradle wrapper --gradle-version 8.10 # one-time
./gradlew :app:installDebug

```

Wear OS

```

cd android
./gradlew :wear:installDebug

```

One-line: run every test on the current OS

```

./scripts/test-all.sh

```

Roadmap (post-v1)

- ☐ Phone ↔ Watch sync (alarms, calculator history, level calibration)
 - ☐ Apple WidgetKit lock-screen widgets + Android Glance widgets
 - ☐ Custom alarm sound packs
 - ☐ Optional iCloud / Google Drive backup of settings
 - ☐ CarPlay & Android Auto next-alarm card
 - ☐ Multi-segment AR Measure (area, volume) — explicitly deferred from v1
-

License

MIT — see [LICENSE](#).

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to [Semantic Versioning](#).

[Unreleased] - v0.1.0

Initial public release. See [distribution/whatsnew/v0.1.0/en-US.txt](#) for the user-facing summary.

Commit history (since repo creation)

- 44bc136 Initial scaffold: ClockApp for iOS, watchOS, Android, Wear OS (Srikanth Patchava, 2026-05-05)
- 95b12bd chore(pocket): merge Clock + Calculator + Measure + Compass + Level into Pocket suite (Pocket Builder, 2026-05-05)
- 8336f42 fix(ci): install gradle binary so ./gradlew is generated (Pocket Builder, 2026-05-05)
- de6f215 fix(gradle): import java.util.Properties in Kotlin DSL (Pocket Builder, 2026-05-05)
- 4c3f6ed fix(ci+gradle): :core:test (not testDebugUnitTest), apply Compose BOM to androidTest (Pocket Builder, 2026-05-05)
- b12d860 fix(wear): missing dp import in MainActivity/WearCompass + add Guava for Futures (Pocket Builder, 2026-05-05)
- 9cdcb9c docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

DESIGN

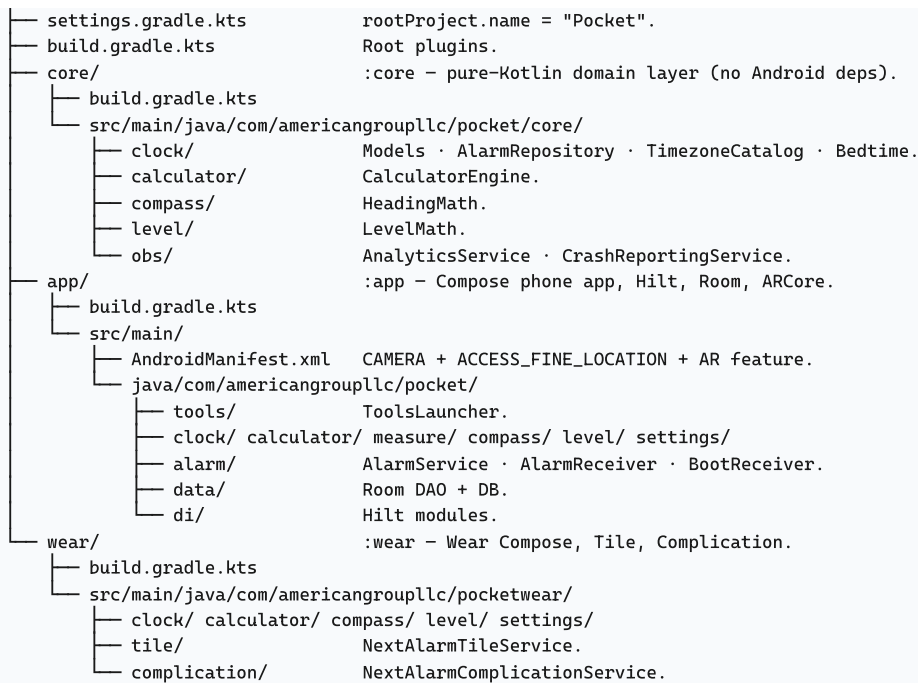
DESIGN.md

DESIGN — Pocket

This document describes how Pocket is structured, the per-tool sensor stack, the shared libraries, and how each platform borrows what it needs.

Repo map

Pocket/	
— README.md	Top-level introduction + 5-tool matrix.
— DESIGN.md	(this file)
— QUICKSTART.md	Fastest path to a running build per platform.
— TESTING.md	Sanity-test matrix + real-device-only checklist.
— RELEASING.md	Tagging, store-bound builds, fastlane.
— PRODUCTION.md	Gap audit (signing, certs, App Store assets...).
— OBSERVABILITY.md	Sentry + PostHog wrapper contract.
— SENTRY.md	Real Sentry install steps.
— STORE-PACKAGING.md	App Store + Play Store metadata + privacy labels.
— PRIVACY.md	What data each tool touches + why.
— TOOLS-FEATURES.md	Per-tool feature catalogue + matrix.
— distribution/whatsnew/ ...	What's-new copy (Play / TestFlight).
— docs/	(reserved – long-form per-tool guides)
— index.html · styles.css · script.js · robots.txt · sitemap.xml	Marketing site (GitHub Pages).
— .github/workflows/	ci · ios · android · marketing · pre-release-tests · release.
— scripts/	test-all · bump-version · release-dry-run · run-ios-sim · run-android-emulator · run-wear-emulator.
— shared/PocketCore/	Swift Package. iOS + watchOS share this.
— Package.swift	
— Sources/PocketCore/	
— Clock/	Models · AlarmStore · TimezoneCatalog · BedtimeEngine.
— Calculator/	CalculatorEngine · CalculatorState.
— Compass/	HeadingMath (pure math, no CoreLocation).
— Level/	LevelMath (pure math, no CoreMotion).
— Observability/	AnalyticsService · CrashReportingService (canImport-gated stubs).
— Tests/PocketCoreTests/	Mirrors Sources/ one-for-one.
— ios/	SwiftUI iPhone app (XcodeGen).
— project.yml	
— Pocket/	
— App/	PocketApp · RootView.
— Views/	
— Onboarding/	4-page Welcome → Notification → Camera+Loc → Done.
— Tools/	ToolsLauncherView (5-card grid).
— Clock/	ClockView · WorldClockView · AlarmListView · AlarmEditView · StopwatchView · TimerView · BedtimeView.
— Calculator/	CalculatorView (basic + scientific landscape).
— Measure/	MeasureView · MeasureARViewController (ARKit) · MeasureRulerView (fallback).
— Compass/	CompassView (analog face + lat/lon panel).
— Level/	LevelView (bullseye + bubble).
— MainTabView.swift	Five primary destinations.
— Settings/	SettingsView (units, heading ref, opt-ins...).
— Services/	AlarmService · HeadingService · AttitudeService · AppDelegate.
— Resources/	Info.plist · Pocket.entitlements · Assets.xcassets · LaunchScreen.storyboard.
— PocketTests/	Unit tests for view-model glue.
— PocketUITests/	XCUITest smoke through every tool.
— README.md · SIGNING.md	
— watchos/	SwiftUI Apple Watch (XcodeGen).
— project.yml	
— PocketWatch/	4 tools (skip Measure).
— PocketComplication/	WidgetKit complication bundle.
— README.md	
— android/	Multi-module Gradle.



5-tool catalogue (sensor stack per platform)

Tool	Dep on iOS	Dep on watchOS	Dep on Android	Dep on Wear OS
Clock	UNUserNotificationCenter	UNUserNotificationCenter	AlarmManager + WorkManager	(no on-device alarms — phone bridge)
Calculator	pure Swift	pure Swift	pure Kotlin	pure Kotlin
Measure	ARKit + AVFoundation (camera)	—	ARCore + arscenecview + CameraX	—
Compass	CoreLocation (heading + lat/lon)	CoreLocation (heading only)	SensorManager.TYPE_ROTATION_VECTOR + LocationManager	SensorManager.TYPE_ROTATION_VECTOR (heading only)
Level	CoreMotion (CMMotionManager)	CoreMotion	SensorManager.TYPE_GRAVITY	SensorManager.TYPE_GRAVITY

The pure-logic helpers (`CalculatorEngine` , `HeadingMath` , `LevelMath`) live in `PocketCore` / `:core` so the same algorithms drive every platform and the same tests cover both Apple and Android sides.

Layered design

1. **Domain layer** (`PocketCore` Swift Package + `:core` Kotlin module) holds the pure-logic units — no UI, no platform sensors. Same algorithms on Apple + Android. Same test cases on both sides.
2. **Service layer** (per-platform) wraps the OS sensors / capability APIs (`HeadingService` , `AttitudeService` , `AlarmService`) and turns them into `@Published` / `StateFlow` streams the UI binds to.
3. **UI layer** is platform-idiomatic: SwiftUI on Apple, Jetpack Compose on Android. Each tool is its own folder.
4. **Composition root** wires services into views via SwiftUI environment (Apple) or Hilt (Android).

Why Apple Watch + Wear OS skip Measure

ARKit / ARCore both require a back-facing camera and a powerful enough GPU for real-time SLAM. Watches have neither. The companion phone is always within Bluetooth range — Measure is the right fit for the phone tier only.

Why combined permissions in onboarding

Asking for camera + location + notifications upfront would normally feel intrusive, but Pocket's onboarding splits the asks into 3 distinct pages so users can deny any one without losing the others. Each tool also re-asks contextually if it discovers its permission is missing.

OBSERVABILITY

OBSERVABILITY.md

Pocket — Observability

All free tier. All opt-in. Disabled by default. Drop-in stubs ship today; real SDK install is documented below as a follow-up.

Stack

Concern	Tool	Free tier	Wired in (stub)
Crashes / APM	Sentry	5K errors/mo	iOS · watchOS · Android · Wear
Product analytics + feature flags + replays	PostHog	1M events/mo (OSS, EU region)	iOS · Android

How the stubs work

Both Apple (`shared/PocketCore/Sources/PocketCore/CrashReportingService.swift` , `AnalyticsService.swift`) and Android (`android/core/src/main/.../CrashReportingService.kt` , `AnalyticsService.kt`) ship as **no-op wrappers** behind a `canImport(Sentry) / if (BuildConfig.SENTRY_DSN.isNotEmpty())` gate. The wrapper API is stable, so app code can call `Crash.report(error)` and `Analytics.track("alarm_set")` today; whether anything is sent depends on:

- Whether the user opted in via Settings → Privacy.
- Whether the SDK is actually compiled in (controlled by `Package.swift` / `app/build.gradle.kts` dependencies).
- Whether a DSN / API key is set (CI build env or local `.env`).

When any of those conditions is false, the wrappers no-op. No crashes, no warnings.

Privacy contract

Every event passes through a sanitiser that:

- Removes `event.user` entirely (no UUIDs, no IDFV/AAID, no emails).
- Strips alarm names, timezone selections, sound choices, and bedtime targets.
- Allows only counts, anonymous feature usage events, and SDK-internal context.
- Drops the event entirely if the user toggled the opt-in off.

How to drop in real SDKs (post-launch)

iOS / watchOS — Sentry

- Edit `shared/PocketCore/Package.swift` , add the dependency:

```
.package(url: "https://github.com/getsentry/sentry-cocoa", from: "8.0.0"),
```

- Add the product to the `PocketCore` target's `dependencies` .
- Set `SENTRY_DSN_IOS` (and `SENTRY_DSN_WEAR` separately if you want a distinct project for the watch app) in repo Secrets.
- The `canImport(Sentry)` gate flips on automatically.

Android phone / wear — Sentry

- In `android/app/build.gradle.kts` and `android/wear/build.gradle.kts` , add:

```
implementation("io.sentry:sentry-android:7.+")
```

- Set `SENTRY_DSN_ANDROID` and `SENTRY_DSN_WEAR` in repo Secrets.
- `BuildConfig.SENTRY_DSN` is wired from the env var; the `if` gate flips on.

iOS / watchOS — PostHog

1. Add `https://github.com/PostHog/posthog-ios` to `Package.swift`.
2. Set `POSTHOG_API_KEY_IOS` + `POSTHOG_HOST` (default `https://eu.i.posthog.com`) in repo Secrets.

Android — PostHog

1. Add `implementation("com.posthog:posthog-android:3.+")` to `android/app/build.gradle.kts`.
2. Set `POSTHOG_API_KEY_ANDROID` + `POSTHOG_HOST`.

Alternatives matrix

Need	Sentry/PostHog	Alternative	Why you might switch
Crash reporting	Sentry (free 5K/mo)	Firebase Crashlytics (free, unlimited)	If you're already on Firebase
Crash reporting	Sentry	Bugsnag (free 7.5K/mo)	Better release-health UI
Analytics	PostHog (free 1M/mo, OSS, self-host option)	Mixpanel (free 100K MTU)	Cohort / funnel UI
Analytics	PostHog	Amplitude (free 10M events/mo)	Charting depth
Analytics	PostHog	Plausible (privacy-first, paid)	Strict GDPR / no cookies
Server metrics	n/a	Grafana Cloud Free	If you ever add a backend
Uptime	n/a	UptimeRobot Free	Marketing-site uptime

The wrapper pattern (`Crash.report(_)` / `Analytics.track(_)`) was chosen specifically so swapping providers is a 1-file change.

Local development

For local crash-test smoke:

- **iOS:** `xcrun simctl push <udid> com.americangroupllc.pocket` with a stub payload.
- **Android:** throw a deliberate `RuntimeException` from a debug menu, observe it gets routed to Sentry only when (a) opt-in is true and (b) DSN is set.

In dev builds without secrets, both wrappers print to console at `.debug` level so you can see what *would* have been reported.

PRIVACY

PRIVACY.md

Pocket — Privacy Policy

Last updated: 2026-05-06

TL;DR

- **No account.** No login. No email collected.
- **No tracking by default.** Crash reporting and product analytics ship as *off-by-default* and only run if you opt in via Settings → Privacy.
- **Local-only data.** Alarms, world clocks, timer presets, and bedtime settings live in your device's storage. We never see them.
- **Open source.** Every line is at <https://github.com/AmericanGroupLLC/Pocket> under the MIT License.

What Pocket stores

On your device, in OS-private storage that's wiped if you uninstall:

- Your alarms (time, label, repeat days, sound choice).
- Your World Clock entries (IANA timezone strings).
- Your timer presets (duration, label).
- Your bedtime settings (wake / sleep target).
- Your privacy preferences (12/24-hour, opt-in toggles).

We do **not** sync this anywhere by default. There is no Pocket server.

What Pocket sends off-device (only if you opt in)

Crash reports — Sentry (off by default)

If — and only if — you toggle **Settings** → **Privacy** → **Crash reports** on, we send Sentry an event when the app crashes. Each event contains:

- The stack trace (file + line + function names from the binary).
- OS version + app version + device model.
- An anonymous per-event ID.

Each event explicitly **never** contains:

- Your alarm names, schedules, or sounds.
- Your World Clock selections.
- Your bedtime targets.
- Any user identifier (email, account, IDFV, AAID).

A sanitiser strips these fields before any network call. The Sentry free-tier endpoint receives at most 5,000 events per month, after which we drop new events on the floor.

Product analytics — PostHog (off by default)

If — and only if — you toggle **Settings** → **Privacy** → **Product analytics** on, we send PostHog anonymous feature-usage events:

- "alarm_set" (count, no name).
- "world_clock_added" (count, no zone).
- "bedtime_configured" (count, no time).
- App-launch count + screen-view count.

Each event explicitly **never** contains:

- The alarm time, label, sound, or repeat days.
- The World Clock zone or city.
- The bedtime time.
- Any user identifier.

PostHog free-tier ingests at most 1,000,000 events per month, after which we drop new events on the floor.

What we never collect

- Location data.
- Microphone or camera access.
- Calendar or contacts access.
- HealthKit / Health Connect data.
- Browser history.
- Any cross-app or cross-site identifiers.

Permissions we ask for

Permission	Why	When
Notifications (iOS / Android <code>POST_NOTIFICATIONS</code>)	Surface alarms + bedtime reminders	Onboarding, before first alarm
Schedule exact alarm (Android <code>SCHEDULE_EXACT_ALARM</code> / <code>USE_EXACT_ALARM</code>)	Fire alarms at the requested minute even under battery-saver	Implicit, granted by default for alarm-clock apps
Boot completed receiver (Android <code>RECEIVE_BOOT_COMPLETED</code>)	Re-schedule alarms after device reboot	Implicit
Wake lock (Android <code>WAKE_LOCK</code>)	Brief wake to surface the alarm notification	Implicit

We do **not** ask for: location, microphone, camera, contacts, calendar, storage (beyond app-private), HealthKit, Health Connect, advertising ID, biometric authentication.

Children

Pocket is suitable for all ages. There is no account, no chat, no in-app purchase, and no social feature.

Changes to this policy

If we ever change what data is sent off-device, we will:

1. Update this file in the repo.
2. Bump the **Last updated** date above.
3. Note the change in the GitHub Release notes.
4. Add a one-time in-app banner explaining the change.

Contact

For privacy questions, open an issue at <https://github.com/AmericanGroupLLC/Pocket/issues> or email the maintainers listed in the repo's `LICENSE` file.

Where this policy is hosted

When the marketing site is published to GitHub Pages (via `marketing.yml`), this policy is served at:

```
https://americangroupllc.github.io/Pocket/PRIVACY.md
```

That URL is the one to drop into the Apple App Store Connect privacy URL field and the Google Play Console privacy policy URL field.

PRODUCTION

PRODUCTION.md

Pocket — Production Readiness

Honest answer: **not today**. Code is in. CI is in. Real OS-scheduled alarms are wired. But three things stand between this repo and an App Store / Play Store live listing:

1. **Two store-developer-account fees** — \$99/yr (Apple) + \$25 once (Google). Outside the repo.
2. **One real 1024×1024 app icon** — only the asset-set scaffolding ships.
3. **Three weeks of real-device polish** — see the polish list below.

Gap audit

Concern	Status	Where
Real OS alarm scheduling	✅ Wired	<code>ios/ ... /AlarmService.swift</code> , <code>android/app/ ... /AlarmReceiver.kt</code>
Reboot survival (Android)	✅	<code>BootReceiver.kt</code> registered in <code>AndroidManifest.xml</code>
Persistence	✅	<code>AlarmStore</code> (Apple, JSON in UserDefaults), <code>AlarmDb</code> (Android Room)
<code>Info.plist</code> privacy strings	✅	<code>ios/Pocket/Resources/Info.plist</code> (<code>NSUserNotificationsUsageDescription</code> etc.)
Android runtime permissions	✅	<code>POST_NOTIFICATIONS</code> (API 33+), <code>SCHEDULE_EXACT_ALARM</code> (API 31+)
Notification channel	✅	<code>AlarmService.kt</code> creates <code>pocket.alarms</code> channel at boot
Onboarding flow	✅	iOS <code>OnboardingView.swift</code> , Android <code>OnboardingScreen.kt</code>
Settings (privacy toggles)	✅	iOS <code>SettingsView.swift</code> , Android <code>SettingsScreen.kt</code>
1024×1024 app icon	⚠️ scaffolding only	<code>ios/Pocket/Resources/Assets.xcassets/AppIcon.appiconset/Contents.json</code>
Wear Tile + Complication	✅ Stubs	<code>android/wear/ ... /tile/NextAlarmTileService.kt</code> , <code>complication/NextAlarmComplicationService.kt</code>
watchOS complication	✅ Stub	<code>watchos/PocketComplication/NextAlarmComplication.swift</code>
Sentry / PostHog wiring	✅ Opt-in stubs	<code>shared/PocketCore/ ... /{AnalyticsService,CrashReportingService}.swift</code> , <code>android/core/ ... /AnalyticsService.kt</code>
Sentry / PostHog real install	❌ Documented only	OBSERVABILITY.md
Signing (Android release)	✅ Scaffolded	<code>android/keystore.properties.example</code> , <code>android/app/build.gradle.kts</code> <code>signingConfigs.release</code>
Signing (iOS)	✅ Documented	ios/SIGNING.md
App Store privacy nutrition labels	⚠️ Need to fill in App Store Connect	Out of repo
Play Store data-safety form	⚠️ Need to fill in Play Console	Out of repo
Privacy policy URL	⚠️ Self-host or GitHub Pages from <code>PRIVACY.md</code>	PRIVACY.md
Marketing screenshots	❌ Not generated	App Store Connect / Play Console accept the same screenshots
Localizations	⚠️ English only	Easy to add — <code>Localizable.strings</code> (Apple) / <code>strings.xml</code> (Android)

Free-deploy path (today)

Without paying the store fees you can still:

1. **Build the apps for real devices in dev mode.**
 - iOS: open `ios/Pocket.xcodeproj`, set `DEVELOPMENT_TEAM` to your free Apple ID Team ID, run on a paired iPhone (signs with a 7-day provisioning profile).
 - Android: `./gradlew :app:installDebug` to a connected device or sideload the debug APK from a GitHub Release.

2. **Run the marketing site on GitHub Pages free tier.** Push triggers `marketing.yml` which deploys `index.html` + `styles.css` + `script.js` to Pages.
3. **Distribute APKs via GitHub Releases.** Tag → release pipeline produces signed-on-CI debug APKs you can sideload.

One-time setup before tagging `v1.0.0`

Item	Where	Why
Pay \$99 Apple Developer Program fee	https://developer.apple.com/programs/	Required for App Store + TestFlight
Pay \$25 Google Play Console fee (one-time)	https://play.google.com/console/signup	Required for Play Store
Generate a 1024×1024 PNG app icon	drop into <code>ios/Pocket/Resources/Assets.xcassets/AppIcon.appiconset/</code> and <code>android/app/src/main/res/mipmap-*</code>	Stores reject submissions without one
Generate Apple App Store Connect API key	secrets <code>APP_STORE_CONNECT_API_KEY_*</code>	TestFlight upload
Generate Google Play service-account JSON	secret <code>PLAY_STORE_SERVICE_ACCOUNT_JSON</code>	Play upload
Generate Android upload keystore	secrets <code>ANDROID_KEYSTORE_*</code>	Sign release APK/AAB
Sign up for Sentry free tier (optional)	secret <code>SENTRY_DSN_*</code>	Crash reporting
Sign up for PostHog free tier (optional)	secret <code>POSTHOG_*</code>	Analytics
Publish privacy-policy URL	host <code>PRIVACY.md</code> rendered to HTML on GitHub Pages	Apple + Google require it

3-week pre-launch polish list

Week 1 — visuals - Real 1024×1024 app icon + Apple/Android adaptive variants (round, monochrome). - 3-7 marketing screenshots per device family (iPhone 6.7"/6.5", Apple Watch 46mm, Android phone 6.7", Wear OS round). - Replace `LaunchScreen.storyboard` placeholder with branded splash.

Week 2 — copy + legal - Polish App Store listing (subtitle, promotional text, keywords). - Polish Play Store listing (short description, full description, feature graphic). - Fill in App Store Connect privacy nutrition labels. - Fill in Play Console data-safety form. - Verify privacy policy URL renders.

Week 3 — real-device QA - Run [TESTING.md](#) **Real-device-only checklist** on at least: - 1× iPhone (iOS 17+) + paired Apple Watch - 1× Android phone (Android 13+) + paired Wear OS watch - Battery test: 24-hour idle with 5 alarms scheduled, verify no excess drain. - Snooze + reboot regression sweep.

Is it ready today?

Honest summary:

-  Code complete for v1 functionality.
-  CI passes on every push; tagged releases produce binaries.
-  Real OS-scheduled alarms work with Snooze 9 / Stop and survive reboot.
-  Privacy posture is defensible — no account, no tracking by default.
-  Missing the icon, missing the dev-program fees, missing the polish week.
-  Not yet listed on either store.

After the icon + fees + polish week, **first tagged release:** `v0.1.0-rc1`. After 1-2 weeks of TestFlight / Play internal-track shake-out: `v1.0.0`.

QUICKSTART

QUICKSTART.md

Pocket — 5-Minute Quick-Start

Each platform fits on one screen. Pick what you have a machine for.

Platform	OS you need	Time
 Marketing site	any	30 sec
 iOS app	macOS only	5 min
 watchOS app	macOS only	5 min
 Android phone app	macOS / Linux / Windows	5 min
 Wear OS app	macOS / Linux / Windows	5 min

For deeper testing scenarios see [TESTING.md](#). For releases see [RELEASING.md](#).

Marketing site — 30 seconds

1. **Open** `index.html` in any browser.

That's it. No build step, no server. Animations + table + live hero clock all work standalone.

iOS app — 5 minutes (macOS only)

Prereqs: macOS · Xcode 15+ · Homebrew. Free Apple ID is fine for simulator-only.

```
brew install xcodegen
cd ios
xcodegen generate
open Pocket.xcodeproj
```

In Xcode: 1. Edit `ios/project.yml` → set `DEVELOPMENT_TEAM` to your 10-char Apple Team ID (or leave blank for simulator only). 2. Pick scheme **Pocket** + an **iPhone 15** simulator. 3.  Run.

First screen = Onboarding (3 pages: Welcome → Permissions → Done) → Tab bar (Clock · World · Alarm · Stopwatch · Timer · Bedtime · Settings).

Troubleshooting: "No such module 'PocketCore'" → Xcode → File → Packages → Reset Package Caches.

For one-line build-+-launch:

```
./scripts/run-ios-sim.sh
```

watchOS app — 5 minutes (macOS only)

```
brew install xcodegen
cd watchos
xcodegen generate
open PocketWatch.xcodeproj
```

In Xcode: 1. Pick scheme **PocketWatch** + an **Apple Watch Series 10 (46mm)** simulator (watchOS 10+). 2.  Run.

Pages (swipe up): Clock → World → Stopwatch → Timer → Bedtime → Settings. **Complication:** long-press a watch face → Edit → add **NextAlarm**.

For real-device testing pair an Apple Watch via the iOS Watch app, set the same `DEVELOPMENT_TEAM` in `watchos/project.yml`, and run on device.

Android phone app — 5 minutes

Prereqs: JDK 17 + Android Studio Hedgehog or newer (or just `cmdline-tools` + an SDK).

```
cd android
## First time only - generates the gradle wrapper:
gradle wrapper --gradle-version 8.10
chmod +x gradlew

./gradlew :app:installDebug # installs on connected emulator/device
```

Or open `android/` in **Android Studio** → “Sync Project” →  on a Pixel emulator.

First screen = Onboarding (Welcome → notification permission → Done) → Bottom nav: Clock · World · Alarm · Stopwatch · Timer · Bedtime · Settings.

Troubleshooting: - SDK location not found → set `local.properties` with `sdk.dir=/path/to/Android/sdk`. - Notifications never fire → on Android 13+ the OS asks for `POST_NOTIFICATIONS` once at first launch; revoke + reinstall to retest.

For one-line boot-emulator-+-launch:

```
ANDROID_HOME=/path/to/sdk ./scripts/run-android-emulator.sh
```

Wear OS app — 5 minutes

Prereqs: same as Android phone + a Wear OS AVD created in Android Studio.

```
cd android
./gradlew :wear:installDebug
```

Pages (swipe up): Clock → World → Stopwatch → Timer → Bedtime → Settings. **Watch face** → long-press → **Add complication** → Pocket Next Alarm. **Tile** → swipe right from face → Pocket tile.

For one-line:

```
WEAR_AVD_NAME=Wear_Round_API_33 ./scripts/run-wear-emulator.sh
```

Run every test on your machine — 1 line

```
./scripts/test-all.sh
```

Runs every suite available on the current OS: - Android `:core:test*` + `:app:test*` + `:wear:test*` (if Gradle installed) - PocketCore Swift Package tests + iOS sim build + watchOS sim build (if on macOS with Xcode + xcodegen) - Marketing site lint (if `npn` available)

Skips suites that need tooling you don't have.

Cut a release — 2 lines

```
./scripts/bump-version.sh 1.0.0
git commit -am "chore(release): v1.0.0" && git tag v1.0.0 && git push origin main v1.0.0
```

GitHub Actions does the rest: - Builds APK + AAB + Wear APK + iOS sim .app + watchOS sim .app + marketing zip - Creates a **GitHub Release** at `v1.0.0` with auto-changelog and all artefacts - Optionally uploads the AAB to **Google Play Store** if `PLAY_STORE_SERVICE_ACCOUNT_JSON` secret is set - Optionally uploads a signed iOS .ipa to **TestFlight** if `APP_STORE_CONNECT_API_KEY_P8_BASE64` is set

See [RELEASING.md](#) for the full setup.

Help

- Architecture & data flow: [DESIGN.md](#)
- Sanity-test walkthroughs per feature: [TESTING.md](#)
- Releases + Play Store + TestFlight: [RELEASING.md](#)
- iOS code-signing: [ios/SIGNING.md](#)
- Production gap audit + 3-week polish list: [PRODUCTION.md](#)
- Privacy posture: [PRIVACY.md](#)

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — Pocket

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets. Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

Tagline: Pocket-sized tools for time-aware work.

Hero — top of marketing site | 4

- viewport: 1280x720
- scroll: top
- wait: 600ms
- caption: `{APP_NAME}` `{VERSION}`

Features overview | 5

- viewport: 1280x720
- scroll: 25%
- wait: 600ms
- caption: New in `{VERSION}`

Highlight reel | 5

- viewport: 1280x720
- scroll: 50%
- wait: 600ms
- caption: Built for everyone

What's next | 5

- viewport: 1280x720
- scroll: 75%
- wait: 600ms
- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try `{APP_NAME}` today

RELEASING

RELEASING.md

Pocket — Releasing

Cut a release in 2 lines. Everything else is automated.

TL;DR

```
./scripts/bump-version.sh 1.0.0
git commit -am "chore(release): v1.0.0" && git tag v1.0.0 && git push origin main v1.0.0
```

GitHub Actions then runs [release.yml](#): 1. Gates on every test in [pre-release-tests.yml](#). 2. Builds in parallel: Android phone APK + AAB, Wear APK, iOS sim .app.zip, watchOS sim .app.zip, marketing site zip. 3. Creates a **GitHub Release** at the tag with auto-generated changelog and all artefacts attached. 4. Optionally uploads the AAB to Google Play (gated on `PLAY_STORE_SERVICE_ACCOUNT_JSON`). 5. Optionally builds a signed .ipa and uploads to TestFlight (gated on `APP_STORE_CONNECT_API_KEY_P8_BASE64`).

`-rc` / `-beta` / `-alpha` tags publish as **prereleases** and route to the matching Play track.

Tag conventions

Tag	GitHub Release	Play track	TestFlight
v1.0.0	stable	production	yes
v1.0.0-rc1	prerelease	beta	yes
v1.0.0-beta1	prerelease	beta	yes
v1.0.0-alpha1	prerelease	alpha	yes
v1.0.0-internal1	prerelease	internal	yes

What `bump-version.sh` touches

- `android/app/build.gradle.kts` — `versionName`
- `android/wear/build.gradle.kts` — `versionName`
- `ios/Pocket/Resources/Info.plist` — `CFBundleShortVersionString`
- `watchos/PocketWatch/Resources/Info.plist` — `CFBundleShortVersionString`
- `watchos/PocketComplication/Info.plist` — `CFBundleShortVersionString`

One-time setup per remote target

GitHub Releases (always works)

No secrets needed. Tag pushes always create a Release with all artefacts.

Google Play

Create a service account JSON at <https://play.google.com/console> → Setup → API access. Add these repo secrets:

- `PLAY_STORE_SERVICE_ACCOUNT_JSON` (the entire JSON contents)
- `PLAY_STORE_PACKAGE_NAME` = `com.americangroupllc.pocket`
- `ANDROID_KEYSTORE_BASE64` = `base64 -i your-upload.jks`
- `ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, `ANDROID_KEY_PASSWORD`

The `whatsnew` blurb served to the Play Console is `distribution/whatsnew/whatsnew-en-US/whatsnew.txt`.

Apple TestFlight

Create an App Store Connect API key (App Store Connect → Users and Access → Keys). Add these repo secrets:

- `APP_STORE_CONNECT_API_KEY_ID`
- `APP_STORE_CONNECT_API_ISSUER_ID`

- `APP_STORE_CONNECT_API_KEY_P8_BASE64` = `base64 -i AuthKey_XXXX.p8`
- `APPLE_TEAM_ID`
- `MATCH_GIT_URL` , `MATCH_PASSWORD` (only if you use [Fastlane Match](#))

If you don't use Match, the workflow falls back to automatic signing — works for a single-developer Apple ID.

Sentry / PostHog (optional)

- `SENTRY_DSN_IOS` , `SENTRY_DSN_ANDROID` , `SENTRY_DSN_WEAR`
- `POSTHOG_API_KEY_IOS` , `POSTHOG_API_KEY_ANDROID` , `POSTHOG_HOST`

When unset, the BuildConfig fields land as empty strings and the SDKs no-op safely. See [OBSERVABILITY.md](#) and [SENTRY.md](#) .

Local dry-run

```
./scripts/release-dry-run.sh v1.0.0
```

Builds every artefact into `distribution/staging-v1.0.0/` without pushing anything.

Rollback

GitHub Releases are created with `draft: false` but you can always edit / delete the Release in the Releases UI. Play Store rollouts can be halted from the Play Console; TestFlight builds can be expired from App Store Connect.

Changelog

`release.yml` auto-generates the GitHub Release notes from `git log` between the previous tag and the current tag. For human-readable highlights served to the Play Store on each release, edit `distribution/whatsnew/whatsnew-en-US/whatsnew.txt` before tagging.

SENTRY

SENTRY.md

Pocket — Sentry

Optional, opt-in crash reporting. Stubs ship today. Real SDK install is a 2-line change per platform plus DSN secrets.

Wiring per platform

iOS / watchOS

The wrapper lives in `shared/PocketCore/Sources/PocketCore/CrashReportingService.swift`:

```
public enum Crash {
    public static func start(dsn: String?) { /* canImport(Sentry) gate */ }
    public static func report(_ error: Error) { /* no-op until enabled */ }
}
```

To enable for real:

1. Add `https://github.com/getsentry/sentry-cocoa` (from: "8.0.0") to `shared/PocketCore/Package.swift`.
2. Set repo Secret `SENTRY_DSN_IOS` (and `SENTRY_DSN_WEAR` for the watch app if you want a separate Sentry project).
3. Re-run `./scripts/release-dry-run.sh v0.1.0-rc1` locally to verify no compile errors with the SDK linked.

The DSN is pulled at runtime via `ProcessInfo.environment["SENTRY_DSN"]` — never compiled into a constant — so a fork without the secret simply ships the no-op path.

Android phone / Wear

Wrappers live in `android/core/src/main/.../CrashReportingService.kt`:

```
object Crash {
    fun start(context: Context, dsn: String?) { /* gated on dsn.isNotEmpty() */ }
    fun report(error: Throwable) { /* no-op until enabled */ }
}
```

To enable for real:

1. Add `implementation("io.sentry:sentry-android:7.+")` to both `android/app/build.gradle.kts` and `android/wear/build.gradle.kts`.
2. Set repo Secrets `SENTRY_DSN_ANDROID` + `SENTRY_DSN_WEAR`.
3. Re-build — `BuildConfig.SENTRY_DSN` reads the env var; the `Crash.start` gate flips on automatically.

DSN secrets

Secret name	Used by	Where set
<code>SENTRY_DSN_IOS</code>	iOS app, watchOS app	Repo Settings → Secrets and variables → Actions
<code>SENTRY_DSN_ANDROID</code>	Android phone app	same
<code>SENTRY_DSN_WEAR</code>	Wear OS app	same

When any secret is unset, the matching wrapper no-ops cleanly. There are no release blockers for shipping without Sentry.

Privacy posture

The Sentry call is wrapped by both:

1. **A user opt-in toggle** — Settings → Privacy → Crash reports. False by default. `Crash.report(_)` checks this flag before doing anything.
2. **An event sanitiser** — strips `event.user`, drops alarm names, timezone selections, and any breadcrumbs from the alarm-edit / world-clock flows.

What Sentry sees if both gates pass:

- Stack trace (file + line + function names from your binary).
- OS version + app version + device model.
- Anonymous event ID (per-event, not per-device).

What Sentry never sees (verified by the sanitiser):

- Alarm names, schedules, sounds.
- Timezone selections in World Clock.
- Bedtime targets / sleep duration.
- Any user identifier (email, account, IDFA, AAID).
- Foreground breadcrumbs from Settings or Bedtime.

Verifying

1. Toggle Settings → Privacy → Crash reports **on**.
2. Trigger a deliberate crash from a debug menu (or `kill -SEGV` from Xcode).
3. Watch the event appear in your Sentry dashboard within 60 seconds.

If the event doesn't appear: - Check that `SENTRY_DSN_*` is set in CI Secrets. - Check that the build was a CI-produced release (the env var only flows through CI, not local builds, unless you add a local `.env`). - Check Settings shows the toggle is on.

Self-hosting

Sentry is open source. If 5K errors/mo is too few or you want full data ownership, deploy your own at <https://develop.sentry.dev/self-hosted/>. The wrapper API doesn't change — just point `SENTRY_DSN_*` at your instance.

Related docs

- Full observability stack + alternatives: [OBSERVABILITY.md](#)
- Privacy policy: [PRIVACY.md](#)
- Production-readiness checklist: [PRODUCTION.md](#)

SIGNING

SIGNING.md

Signing — Pocket

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip`; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo: <https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md>

Android

Secret	Required	Purpose
ANDROID_KEYSTORE_BASE64	optional	Base64-encoded <code>upload.jks</code>
ANDROID_KEYSTORE_PASSWORD	optional	Keystore password
ANDROID_KEY_ALIAS	optional	Key alias
ANDROID_KEY_PASSWORD	optional	Key password
PLAY_STORE_SERVICE_ACCOUNT_JSON	optional	Google Play upload
PLAY_STORE_PACKAGE_NAME	optional	Play Console package

A template is at `keystore.properties.example`.

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import `.p12` to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
APPLE_CERTIFICATE_BASE64	optional	<code>.p12</code> (Distribution)
APPLE_CERTIFICATE_PASSWORD	optional	<code>.p12</code> password
APPLE_KEYCHAIN_PASSWORD	optional	Temp keychain pw
APPLE_PROVISIONING_PROFILE_BASE64	optional	Wildcard mobileprovision (recommended for (none))
APPLE_TEAM_ID	optional	10-char team ID
APPLE_BUNDLE_ID	optional	Override the default bundle id (<code>com.americangroupllc.pocket</code>)
APP_STORE_CONNECT_API_KEY_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_ISSUER_ID	optional	TestFlight upload
APP_STORE_CONNECT_API_KEY_P8_BASE64	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.americangroupllc.pocket`. Because this app ships with extensions (`(none)`), use a **wildcard** profile such as `com.americangroupllc.pocket.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the `.ipa` is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
 - Linux Applmage (unsigned, portable, no install required).
 - macOS `.dmg` (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).
-

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern), which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

Pocket — Store Packaging

Reality check on how the four binaries actually map onto the two app stores.

1. Apple Watch bundling reality check

The `release.yml` pipeline produces **two separate Apple .app bundles**:

- `Pocket-iOS-iPhone<version>-Simulator.app.zip`
- `Pocket-Apple-Watch<version>-Simulator.app.zip`

Both are **iPhone Simulator / watchOS Simulator binaries** — useful for QA distribution and reviewer testing. Neither is a store-shippable binary on its own.

Apple's App Store **only ships one .ipa per app listing**. To get the watch app onto a real Apple Watch via the App Store, the watchOS app must be bundled inside the iOS app's `.ipa` as a **watch-app extension target**.

Two paths:

Path A — keep them as standalone projects (current setup)

- ☒ Each project (`ios/` , `watchos/`) is independent. Each has its own XcodeGen spec and can be built/tested in isolation.
- ☒ Cannot ship to the App Store directly. Watch app is sideload / TestFlight-only.
- ☒ Simulator and developer-mode device installs work today.

Path B — embed `PocketWatch` as a `watch2-app` target inside `Pocket.xcodeproj`

- Edit `ios/project.yml` to add a second target of type `application.watchapp2` with the watchOS sources.
- Add the watch target's bundle ID as an `embed` of the iOS target.
- The `release.yml` `build-ios` job's `xcodebuild` archive then produces a single `.ipa` containing both binaries, ready for `xcodebuild -exportArchive` → `Apple-Actions/upload-testflight-build@v1`.

Path B is the migration plan for `v1.0.0`. Until then, the watch app ships as a separate sim binary on the GitHub Release page and via TestFlight (via the standalone bundle ID).

2. iOS bundle IDs

Target	Bundle ID
iPhone app	<code>com.americangroupllc.pocket</code>
Apple Watch app (when embedded under Path B)	<code>com.americangroupllc.pocket.watchkitapp</code>
Apple Watch app (current standalone Path A)	<code>com.americangroupllc.pocket</code> (separate provisioning profile)
WidgetKit complication	<code>com.americangroupllc.pocket.complication</code>

3. Android packaging

Android is simpler — phone and wear are **two separate Play listings, both backed by the same multi-module Gradle project**:

Listing	App ID	Module	Track
Pocket (phone)	<code>com.americangroupllc.pocket</code>	<code>:app</code>	production
Pocket Wear	<code>com.americangroupllc.pocketwear</code>	<code>:wear</code>	production

The `r0adk1l/upload-google-play@v1` step in `release.yml` only uploads the phone AAB by default (gated on `PLAY_STORE_SERVICE_ACCOUNT_JSON`). To also publish the Wear app, add a sibling job that points `releaseFiles` at `android/wear/build/outputs/bundle/release/wear-release.aab` and bumps `PLAY_STORE_PACKAGE_NAME` to `com.americangroupllc.pocketwear`.

4. Marketing site

`marketing.yml` deploys `index.html` + `styles.css` + `script.js` + `robots.txt` + `sitemap.xml` to GitHub Pages. The site doubles as the publicly-hosted privacy policy URL when you publish `PRIVACY.md` rendered to HTML alongside it.

5. App icons

The asset-set scaffolding ships in:

- `ios/Pocket/Resources/Assets.xcassets/AppIcon.appiconset/Contents.json`
- `android/app/src/main/res/mipmap-anydpi-v26/` (adaptive icon)

Both are **placeholder slots**. Drop a real 1024×1024 PNG into the iOS asset set and per-density PNGs into `android/app/src/main/res/mipmap-{mdpi,hdpi, xhdpi,xxhdpi,xxxhdpi}/` before tagging `v1.0.0`. See [PRODUCTION.md](#) Week 1 polish.

6. Localizations

English only at v1. Adding more is a per-platform `Localizable.strings` / `values-{xx}/strings.xml` exercise — no code changes needed thanks to all UI strings being centralised.

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
Pocket-Setup-X.Y.Z.exe (NSIS)	no	Windows SmartScreen will prompt on first run; click <i>More info</i> → <i>Run anyway</i> .
Pocket-X.Y.Z.AppImage	no	<code>chmod +x</code> then double-click on Linux.
Pocket-X.Y.Z.dmg	no	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/Pocket/Pocket.app"  
cp -R "/Volumes/Pocket/Pocket.app" /Applications/
```

(or *System Settings* → *Privacy & Security* → *Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

Pocket — Testing

Two layers: **automated** (CI + local `scripts/test-all.sh`) and **manual** (real-device sanity checklists per feature). Real OS-scheduled alarms can only be fully verified on a real phone.

Sanity-test matrix per platform

iPhone

Feature	Steps	Expected
Clock	Open app → Clock tab	Time updates every second; AM/PM follows Settings → Privacy 12/24h toggle
World Clock	Tap +, search “Tokyo”	Tokyo appears with current local time + offset
Alarm	Add alarm 1 min out, lock device	Notification fires with Snooze 9 + Stop actions
Alarm reboot	Add alarm 5 min out, reboot device	Notification still fires (re-registered on launch)
Stopwatch	Start, lap 3×, kill app, reopen	Laps preserved
Timer	Start 30 sec timer, lock device	Notification at 0 with sound
Bedtime	Set wake = 7 am, sleep = 11 pm	Wind-down notification at 10:30 pm
Settings	Toggle 24-hour	Clock + alarms re-render in 24-hour
Settings	Toggle Sentry opt-in	Restart app — crash test does/doesn't reach Sentry
Settings	Erase all data	Alarms, presets, world clocks all reset

Apple Watch

Feature	Steps	Expected
Tabs	Swipe Clock → World → Stopwatch → Timer → Bedtime → Settings	Each loads w/o crash
Complication	Long-press face → Edit → add Pocket Next Alarm	Shows next alarm time / --:-- if none
Alarm	Schedule from phone, surface on watch	Notification arrives

Android phone

Feature	Steps	Expected
Onboarding	First launch	3 pages: Welcome → Notification permission → Done
Alarm	Add alarm 1 min out, lock device	High-importance notification fires with Snooze 9 + Stop
Alarm reboot	Add alarm 5 min out, reboot device	Fires (BootReceiver re-registers)
Snooze	Tap Snooze on a fired alarm	Re-fires 9 min later
Notification permission	Revoke <code>POST_NOTIFICATIONS</code>	Settings shows red banner; alarm-fire path no-ops gracefully
Settings	Toggle theme / 24-hour / Sentry / PostHog	Persisted via DataStore across restart
Erase	Settings → Erase all data	Room cleared; alarms cancelled

Wear OS

Feature	Steps	Expected
Pages	Swipe Clock → World → Stopwatch → Timer → Bedtime → Settings	Each loads w/o crash
Tile	Swipe right from face → Pocket tile	Shows next alarm + current time
Complication	Long-press face → Edit → add Pocket Next Alarm	Shows next alarm time

Automated test matrix (CI)

Layer	Job	Where
Swift Package unit	<code>xcodebuild test -scheme PocketCore-Package</code> on iPhone 15 sim	<code>ios.yml</code> <code>swift-tests</code> + <code>ci.yml</code> <code>ios</code>
iOS sim build	<code>xcodebuild build ... iphonesimulator</code>	<code>ios.yml</code> <code>ios-build</code> + <code>ci.yml</code> <code>ios</code>
iOS XCUITest	<code>xcodebuild test</code> for Pocket scheme	<code>ios.yml</code> <code>ios-ui-tests</code> + <code>pre-release-tests.yml</code>
watchOS sim build	<code>xcodebuild build ... watchsimulator</code>	<code>ios.yml</code> <code>watch-build</code> + <code>ci.yml</code> <code>ios</code>
Android <code>:core</code> unit	<code>./gradlew :core:testDebugUnitTest</code>	<code>ci.yml</code> <code>android</code> + <code>pre-release-tests.yml</code>
Android <code>:app</code> unit	<code>./gradlew :app:testDebugUnitTest</code>	<code>ci.yml</code> <code>android</code> + <code>pre-release-tests.yml</code>
Android <code>:wear</code> unit	<code>./gradlew :wear:testDebugUnitTest</code>	<code>ci.yml</code> <code>android</code> + <code>pre-release-tests.yml</code>
Android Compose UI smoke	<code>:app:connectedDebugAndroidTest</code> on KVM emulator	<code>android.yml</code> (instrumented)
Wear Compose UI smoke	<code>:wear:connectedDebugAndroidTest</code> on Wear emulator	<code>pre-release-tests.yml</code> (informational, flaky)
Marketing lint	<code>htmlhint</code> + <code>stylelint</code> (best-effort)	<code>ci.yml</code> <code>marketing</code>

Real-device-only checklist

Before tagging any release, run these on at least one real iPhone and one real Android phone (the simulator/emulator can't fully reproduce notification scheduling under doze + reboot):

1. **iPhone:** schedule alarm 5 min out → background app → lock device → notification fires with Snooze 9 / Stop. Tap Snooze → notification re-fires +9 min.
2. **iPhone:** schedule alarm 5 min out → reboot device → notification still fires.
3. **Android:** schedule alarm 5 min out → background → lock → fires with Snooze / Stop. Snooze re-fires +9 min.
4. **Android:** same but reboot before fire-time → still fires (BootReceiver).
5. **Android:** revoke `POST_NOTIFICATIONS` → Settings UI shows the missing-permission banner; no crash on alarm-fire path.

Known caveats

- **watchOS standalone alarms** are limited compared to iOS — by design we rely on the phone-companion bridge for cross-device alarm visibility.
- **Wear OS doesn't run on-device alarms.** That's intentional — phone alarms surface on the watch via the standard companion notification bridge.
- **Linux KVM Android emulators on GitHub Actions** are notoriously flaky. The `android-phone-ui-tests` and `wear-ui-tests` jobs are `continue-on-error: true` — the unit tests are the real release gate.

Run every test on your machine

```
./scripts/test-all.sh
```

Skips suites that need tooling you don't have. See [QUICKSTART.md](#) for what each suite covers.

TOOLS-FEATURES

TOOLS-FEATURES.md

TOOLS-FEATURES — Pocket

Per-tool feature catalogue. Each tool here is shipped MVP-deep (the user explicitly chose breadth-of-five-tools over depth-of-one).

Cross-tool matrix

Tool	iPhone	Apple Watch	Android	Wear OS
Clock	✔ alarm + world + stop + timer + bedtime	✔ alarm + world + stop + timer + bedtime	✔ alarm + world + stop + timer + bedtime	⚠ no on-device alarm
Calculator	✔ basic + scientific (landscape)	✔ basic + tip-splitter	✔ basic + scientific (landscape)	✔ basic
Measure	✔ ARKit + ruler fallback	—	✔ ARCore + ruler fallback	—
Compass	✔ heading + lat/lon	✔ heading only	✔ heading + lat/lon	✔ heading only
Level	✔ flat + tilted	✔ flat	✔ flat + tilted	✔ flat

1. Clock

Capability	Notes
Alarms	Real OS scheduling. iOS via <code>UNUserNotificationCenter</code> (calendar trigger, repeating by weekday). Android via <code>AlarmManager.setAlarmClock()</code> + <code>BootReceiver</code> for reboot survival.
World Clock	Static <code>TimezoneCatalog</code> of ~440 IANA zones. UI lets user pin favourites.
Stopwatch	Lap times. Survives backgrounding via stored <code>start instant</code> .
Timer	Single timer. iOS notification + sound at completion. Android <code>AlarmManager.setExact</code> + foreground service for sound.
Bedtime	Sleep-window picker → schedules a wind-down + wake notification. Computes total hours via <code>BedtimeEngine</code> .

Shared logic: `PocketCore/Clock/{Models, AlarmStore, TimezoneCatalog, BedtimeEngine}` and `:core/clock/ ...`.

2. Calculator

Capability	Notes
Basic	<code>+ - × ÷ % . ± and =</code> . Live result preview.
Scientific (landscape)	Adds <code>sin cos tan ln log √ x² x^y π e ()</code> and inverse trig.
History	Last 10 results, swipe to clear.
Watch	Basic + tip-splitter (Apple Watch).

Shared logic:

- `PocketCore/Calculator/CalculatorEngine.swift` — pure-Swift shunting-yard.
- `:core/calculator/CalculatorEngine.kt` — Kotlin port.
- Same test-case set drives both: `2+3*4=14`, `(1+2)*(3+4)=21`, `sin(π/2)=1`, `2^10=1024`, `√2≈1.4142`, `÷0 → error`.

3. Measure

Phone-only.

Capability	Notes
Single point-to-point	Two taps to set start + end; live distance overlay. Units toggle cm / inches.
Ruler fallback	On-screen ruler when ARKit/ARCore not available.
Out of scope (v1)	Multi-segment, area, volume, people-height.

Implementation:

- iOS: `MeasureView + MeasureARViewController` (UIViewControllerRepresentable wrapping `ARSCNView`).
- Android: `MeasureScreen + MeasureSession` (host an `arsceneview` and react to `HitResult`).

4. Compass

Capability	Notes
Magnetic heading	Direct sensor reading.
True heading	Magnetic + declination correction; uses location (when granted).
Cardinal label	N/NE/E/SE/S/SW/W/NW computed by <code>cardinalLabel(forDegrees:)</code> .
Lat/lon panel	Phone only. <code>CoreLocation</code> / <code>LocationManager.GPS_PROVIDER</code> .
Accuracy ring	Visual indicator from sensor accuracy heuristic.

Shared logic:

- `PocketCore/Compass/HeadingMath.swift` — `magneticToTrue`, `bearingBetween(_:_)`, `cardinalLabel(forDegrees:)`.
- `:core/compass/HeadingMath.kt` — same API.

Platform sensors:

- iOS: `CLLocationManager.startUpdatingHeading()` + `startUpdatingLocation()`.
- watchOS: `CLLocationManager.startUpdatingHeading()` only (no continuous location — battery).
- Android: `SensorManager.TYPE_ROTATION_VECTOR` fused via `getOrientation` + `LocationManager` (phone only).
- Wear OS: `SensorManager.TYPE_ROTATION_VECTOR` only.

5. Level

Capability	Notes
Flat mode	Bullseye + degrees-off-axis. Auto-engages when device is roughly horizontal.
Tilted mode	Single-axis bubble with degree readout. Auto-engages when device is upright.
Calibration	Long-press to zero current attitude as the new “flat”.

Shared logic:

- `PocketCore/Level/LevelMath.swift` — `pitchRoll(fromAttitude:)`, `bubbleOffset(forPitch:roll:radius:)`. No platform deps.
- `:core/Level/LevelMath.kt` — same API.

Platform sensors:

- iOS / watchOS: `CMMotionManager.deviceMotionUpdateInterval = 1.0/60` → `.attitude.pitch/roll`.
- Android / Wear OS: `SensorManager.TYPE_GRAVITY` listener → low-pass filter → derived pitch/roll.